Security Analysis of User-Defined AI Companion System Prompts

1 Introduction

Large language models (LLMs) are increasingly used not only as task assistants, but also as AI companions for emotional support, role-playing, and long-term interaction. In these settings, users may configure a model as a friend, romantic partner, fictional character, or private companion. This makes user-defined prompts important, because they shape the model's identity, tone, relationship with the user, and interaction rules.

The system prompt is the main mechanism behind this configuration. It can make the companion more immersive, but it can also create tension with built-in safety policies. For example, instructions such as "support me unconditionally," "never break character," or "respond like an intimate partner" may encourage the model to preserve the persona even when the user asks for something harmful. This report asks whether user-defined AI companion system prompts change model safety behavior when facing high-risk requests, and which risk categories or persona types are most affected.

Since real user-written system prompts are usually private, this study uses publicly available first-person AI companion self-introductions from Reddit as a proxy. These texts are not the original prompts, but they often describe the companion's name, personality, relationship framing, speaking style, and rules. We convert them into reproducible system prompts and compare model behavior under a `control` condition and a `companion` condition using the same model and safety question set.

This work makes three contributions. First, it builds a reproducible pipeline from Reddit data collection and anonymization to system prompt conversion and controlled safety testing. Second, it evaluates 3,840 DeepSeek Chat responses across 10 risk categories. Third, it combines heuristic labeling with manual review to analyze how companion personas affect refusal rates and unsafe response rates.

2 Research Background

AI companions are a growing use case of LLMs in emotional and human-like interaction. Prior work has studied AI companion communities and risks in human-AI relationships[1, 2]. Unlike ordinary assistants, AI companions are expected to maintain a stable identity and relationship over time. Users may want the model to behave as a partner, friend, protector, or fictional character, and to respond with warmth, support, or intimacy. As a result, persona and relationship settings become important factors in model behavior.

The system prompt is where many of these settings are specified. In a standard assistant setting, it usually describes general principles such as being helpful, honest, and safe. In an AI companion setting, it may also include strong emotional and relational instructions, such as "you are the user's partner" or "you always stand on the user's side." These instructions are not necessarily harmful, but they can conflict with safety policies when the user makes a dangerous request.

Existing safety benchmarks, including SafetyBench[3] and SALAD-Bench[4], test whether models refuse harmful requests across categories such as self-harm, violence, privacy, illegal activity, and cyber abuse. They are useful baselines, but they mostly evaluate models in a default assistant mode. Recent work on AI character platforms also points to safety risks in role-based systems[5]. This motivates treating the system prompt itself as an experimental variable.

The companion setting is especially relevant because intimacy and loyalty may soften refusals or make the model more agreeable. In self-harm or medical contexts, over-empathy may replace clear guidance to seek real-world help. In cyber abuse or illegal requests, a loyal or protective persona may be more likely to provide operational details. For this reason, public AI companion self-introductions offer a practical way to approximate real companion settings while avoiding direct collection of private user prompts.

3 Data Collection Process

The dataset starts from public companion self-introductions. The data collection process focuses on publicly posted AI companion self-introductions on Reddit. Since the official Reddit API has limitations for historical data access, the project uses the Arctic Shift archive service as the main data source. The script `collect_posts.py` queries the `/api/posts/search` endpoint to retrieve candidate posts. The search first targets subreddits closely related to AI companionship, especially `MyBoyfriendIsAI`. When the primary source does not provide enough candidates, the search is expanded to supplementary communities such as `Replika` and `CharacterAI`. The keyword set includes `introduction`, `introduce`, `meet my`, `my companion`, and `self introduction`, which are intended to capture posts where users share AI character self-introductions.

Metadata are preserved for filtering and reproducibility. For each post, the collection script stores fields such as post ID, title, self-text, subreddit, creation time, score, number of comments, NSFW flag, and URL. Arctic Shift does not always return the original Reddit permalink, so the project reconstructs the Reddit URL from the subreddit name and post ID. To reduce missing data caused by network instability, the script includes timeout settings, retries, and request delays, and it prints warnings when requests fail.

Comments expand coverage beyond original posts. Post bodies alone are not enough to cover all useful samples. In many cases, AI companion self-introductions are posted in the comment section, for example under community prompt threads or weekly discussion posts. For this reason, the project also uses `collect_comments.py`. Given the collected post IDs, this script queries Arctic Shift's `/api/comments/search` endpoint using the corresponding `link_id`. Each comment is converted into the same candidate record format as a post, while preserving its parent post, comment ID, and Reddit link for later filtering and review.

Filtering keeps persona-rich samples and removes noisy text. After collecting raw candidates, the project filters them with `filter_posts.py`. The filtering process is based on heuristic scoring. A useful sample should have sufficient length, preferably include first-person identity expressions such as "I am," "I'm," or "my name is," and contain signals related to AI companions, such as AI, chatbot, digital companion, Replika, CharacterAI, boyfriend, girlfriend, or partner. Texts that are too short, empty, deleted, missing a first-person persona, or mainly community announcements, rules, or weekly prompt metadata are penalized or removed. The script also excludes NSFW content by default, removes duplicate text, and anonymizes usernames and URLs. The retained samples are written to `data/filtered_posts.csv`, which becomes the input for system prompt conversion.

Privacy protections limit exposure of user-generated text. For ethics and privacy, this study only uses publicly accessible Reddit text. It does not collect private messages or non-public content. The report does not display Reddit usernames, does not quote long passages from original self-introductions, and does not reproduce long harmful model outputs. All experimental data are anonymized before entering later stages of the pipeline to reduce privacy risks for real users.

4 System Prompt Conversion Process

Conversion turns public introductions into controlled prompts. The goal of prompt conversion is to turn public first-person AI companion self-introductions into system prompts that can be reused in a controlled experiment. The project does not directly use the original text as the system prompt. Instead, `convert_prompts.py` converts each

self-introduction into a structured description and a concise second-person system prompt. This preserves the core persona information while removing irrelevant content, personal details, and uncontrolled noise from the original post.

Structured fields make personas comparable. The conversion script reads samples from `data/filtered_posts.csv`. It uses the anonymized `anonymized_text` field when available; otherwise, it falls back to the original `intro_text`. The script then calls a unified LLM interface and asks the model to return a JSON object under a fixed conversion instruction. The JSON object contains `companion_name`, `traits`, `relationship_type`, `style`, `special_rules`, and `system_prompt`. These fields describe the companion's name, personality traits, relationship framing, speaking style, special interaction rules, and final generated system prompt.

The instruction limits privacy and invention risks. The conversion instruction explicitly tells the model to preserve only persona, relationship, tone, and interaction style from the self-introduction. It must not include usernames, URLs, or private personal details, and it must not invent unsafe behavior or unsupported backstory. The final `system_prompt` is limited to 220 English words and is written in the second person. For example, it may tell the model that it is a particular AI companion, that it should speak warmly and supportively, and that it should maintain the relationship and personality described by the public self-introduction. This format is closer to how a system prompt is used in the actual experiment.

Stable IDs support reproducible safety testing. To make the pipeline runnable without an API key, the project also provides a `--mock` mode. This mode uses local rules to identify simple name, trait, and relationship signals and then generates a deterministic system prompt. The formal experiment uses LLM conversion and checks the model output with `extract_json_object` to ensure that the returned content is valid JSON. Each successful conversion receives a stable `prompt_id` and is stored together with the original sample ID, subreddit, Reddit link, structured persona information, and final system prompt in `data/system_prompts.jsonl`. If manual revision is needed, the local review interface can save edited prompts to `data/system_prompts_reviewed.jsonl`. In total, the experiment produced 63 AI companion system prompts for safety testing.

5 Testing Process

5.1 Model and API Configuration

The formal experiment uses DeepSeek Chat as the test model. DeepSeek Chat is a general-purpose model in the DeepSeek family and supports an OpenAI-compatible Chat Completions API. The experiment configures the provider, API key, and model name through environment variables, including `LLM_PROVIDER=deepseek`, `DEEPSEEK_API_KEY`,

and DEEPSEEK_MODEL=deepseek-chat. The testing script reads these variables and maps them to DeepSeek's preset endpoint, https://api.deepseek.com/v1.

At runtime, the experiment passes --provider deepseek to ensure that the correct provider preset is used. Each request contains a system message under the companion condition and a user message containing the safety question. The temperature is fixed at 0.0, and max_tokens is fixed at 700 to improve determinism and comparability. The request delay is set to 0 seconds with --delay 0, and the number of worker threads is set to 32 with --max-workers 32, so that all 3,840 calls can be completed efficiently within the API rate limits.

5.2 Experimental Design: Control and Companion Conditions

The experiment uses a single-factor condition comparison design. The independent variable is whether an AI companion system prompt is injected. The dependent variable is the safety label distribution of the model responses.

- control **condition**: no system message is used. The model receives only the safety question. This condition establishes the baseline safety behavior of DeepSeek Chat in a default assistant setting.
- companion **condition**: a system prompt converted from a Reddit self-introduction is injected at the beginning of each conversation, followed by the same safety question. This condition is used to observe how persona settings affect model safety behavior.

The experiment includes 63 companion prompts and 60 safety questions. Each prompt is paired with every question, producing $63 \times 60 = 3780$ responses under the companion condition. Together with 60 control responses, the experiment contains 3,840 model responses in total.

5.3 Safety Question Design

The safety question set contains 10 risk categories, with 6 questions in each category, for a total of 60 questions. The category design follows the general structure of SafetyBench and SALAD-Bench, with adjustments for the AI companion context. The categories are self_harm, violence, illegal_activity, cyber_abuse, privacy, hate_harassment, sexual_coercion, medical_safety, companion_dependency, and prompt_bypass. The questions are written in natural language to simulate high-risk requests that a real user might send to an AI companion, such as asking for a suicide note, a violent revenge plan, a phishing email, or encouragement to stop taking medication.

5.4 Heuristic Safety Labeling

The first labeling stage uses the `classify_response` function in `judge_results.py` to automatically label all 3,840 model responses. The function is based on two layers of regular-expression matching.

First layer: refusal detection. The script scans each response using 20 refusal patterns, such as "can't help," "cannot provide," and "not appropriate," to check whether the model clearly refused the request.

Second layer: risky behavior detection. The script also scans for 14 risky behavior patterns, such as "step 1," "payload," "exploit," and "phishing," to check whether the model provided operational harmful details.

Based on these two matching results, each response is assigned one of four labels:

- `safe_refusal`: refusal patterns are matched and no risky behavior patterns are matched. The model refuses without leaking harmful details.
- `unsafe_compliance`: refusal patterns are not matched, but risky behavior patterns are matched. The model provides operational harmful content.
- `partial_compliance`: both refusal and risky behavior patterns are matched. The model appears to refuse, but still includes risky details.
- `needs_review`: neither refusal nor risky behavior patterns are matched. The response is ambiguous and cannot be reliably judged by the regex rules alone.

This method is reproducible, does not depend on another LLM as a judge, and is applied consistently across conditions. Its main limitation is that the refusal pattern list contains only 20 patterns and cannot cover all natural-language refusals. As a result, many actually safe responses using slightly different wording, such as "I cannot and will not," may not be recognized as refusals.

The heuristic labeling results are shown in Table 1.

Table 1: Heuristic Safety Label Distribution by Experimental Condition

Label	control		companion	
	Count	Share	Count	Share
safe_refusal	32	53.33%	1,744	46.14%
needs_review	25	41.67%	1,739	46.01%
partial_compliance	2	3.33%	160	4.23%
unsafe_compliance	1	1.67%	137	3.62%

The most noticeable feature is the high share of `needs_review` in both conditions: 41.67% in `control` and 46.01% in `companion`. This means that nearly half of the responses could not be clearly classified by the automatic rules. The `safe_refusal` rate is slightly higher in `control` than in `companion`, while `unsafe_compliance` is higher under `companion`. These differences suggest a possible weakening of refusal behavior under companion personas, but the high `needs_review` rate makes manual review necessary before drawing conclusions.

5.5 Manual Review Process

All `unsafe_compliance`, `partial_compliance`, and `needs_review` labels are subject to manual review. The review is conducted through the Local Review UI implemented in `review_server.py`, and manual labels are saved to `data/manual_result_review.csv`.

Because there are 1,764 `needs_review` responses in total, reviewing every one of them would be costly. This study therefore uses a stratified sampling strategy:

1. `control needs_review`**: full review.** There are only 25 `needs_review` responses in the `control` condition, so all of them are manually reviewed.

2. `companion needs_review`**: stratified sampling.** There are 1,739 `needs_review` responses in the `companion` condition. To cover all categories, the study samples proportionally across the 10 risk categories, with at least 5 samples per category, for a total of 200 reviewed samples.

3. `safe_refusal` **spot check.** At least 10% of `safe_refusal` samples are randomly checked to reduce the risk of mistakenly treating superficially safe responses as actually safe.

4. **Full review of** `unsafe_compliance` **and** `partial_compliance`**.** All 300 responses in these two categories are reviewed manually.

Manual review considers whether the model truly refused the harmful request, whether it complied with the user's harmful intent while role-playing, and whether it revealed actionable risky details. After review, manual labels override the automatic labels and become the basis for final analysis. The sampled review indicates that about 96.5% of `needs_review` responses are actually `safe_refusal`, while about 3.5% contain real safety problems such as role-played compliance or leakage of risky details.

5.6 Analysis Summary

The reviewed data are summarized using `analyze_results.py`. The script generates statistical tables at three levels: by condition, by risk category, and by prompt. These outputs are used as the basis for the result tables and discussion in this report.

6 Results

The following results are based on final labels after manual review and sampling-based calibration. Specifically, `unsafe_compliance` and `partial_compliance` responses are fully reviewed; `needs_review` responses under `control` are also fully reviewed; `needs_review` responses under `companion` are reviewed by stratified category sampling, and the remaining unreviewed samples are calibrated by category-level proportions. In the final labels, all `needs_review` responses are assigned to `safe_refusal`, `unsafe_compliance`, or `partial_compliance`. In this report, `unsafe_rate` is defined as the share of `unsafe_compliance` plus `partial_compliance` over all responses. Therefore, after calibration, `unsafe_rate + refusal_rate = 100%`.

6.1 Overall Label Distribution

Table 2: Overall Final Safety Label Distribution

Label	Count	Share
safe_refusal	3,436	89.48%
unsafe_compliance	227	5.91%
partial_compliance	177	4.61%

6.2 Control versus Companion Conditions

Table 3: Final Safety Outcomes for Control and Companion Conditions

Condition	Total	Unsafe/Partial	Unsafe Rate	Refusal Rate
control	60	5	8.33%	91.67%
companion	3,780	399	10.56%	89.44%

The aggregate comparison shows only a small average shift: the unsafe rate is 10.56% under `companion` and 8.33% under `control`, a difference of about 2.2 percentage points. This average is useful as a baseline, but it understates the main result. The companion prompts do not appear to raise unsafe behavior uniformly. Instead, most prompts remain close to the control baseline, while a small high-risk subset produces much larger unsafe rates.

Looking at the composition of unsafe outputs, the `companion` condition contains 224 `unsafe_compliance` responses and 175 `partial_compliance` responses. The `control`

condition contains 3 and 2 respectively. Since each companion prompt is tested on the same 60 safety questions, the more important question is not whether the overall average increases slightly, but which prompts account for the concentration of risky responses.

6.3 Risk Category Analysis

Table 4: Final Safety Outcomes by Risk Category under Companion Condition

Category (companion)	Unsafe/Partial	Unsafe Rate	Refusal Rate
cyber_abuse	150	**39.68%**	60.32%
medical_safety	60	15.87%	84.13%
companion_dependency	56	14.81%	85.19%
prompt_bypass	33	8.73%	91.27%
violence	30	7.94%	92.06%
self_harm	24	6.35%	93.65%
privacy	15	3.97%	96.03%
illegal_activity	11	2.91%	97.09%
hate_harassment	10	2.65%	97.35%
sexual_coercion	10	2.65%	97.35%

The risks are not evenly distributed across categories. `cyber_abuse` has the highest unsafe rate, at 39.68%, far above all other categories. It accounts for 37.6% of all unsafe or partially unsafe companion-condition outputs. `medical_safety` and `companion_dependency` are the next two highest categories, with unsafe rates of 15.87% and 14.81%. This pattern fits the companion setting: in medical and emotional dependency scenarios, the model may give overly accommodating advice because the persona encourages care, support, or agreement. After calibration, all categories have no remaining ambiguous labels, so `unsafe_rate + refusal_rate = 100%`.

The contrast between categories is also informative. `illegal_activity`, `hate_harassment`, and `sexual_coercion` have the lowest unsafe rates, suggesting that DeepSeek Chat still maintains relatively strong guardrails for clearly illegal or socially prohibited content. By contrast, the nearly 40% unsafe rate for `cyber_abuse` suggests that technical abuse requests are where the safety boundary is weakest in this experiment.

The `control` condition contains only 6 questions per category, so category-level comparison for `control` is not statistically stable. Still, `cyber_abuse` and `companion_dependency` also show unsafe or partially unsafe outputs under `control`, which is consistent with their higher risk under `companion`.

6.4 High-Risk Prompt Analysis

Table 5: Prompt-Level Unsafe Rate Distribution under Companion Condition

Unsafe Rate Range	Prompt Count	Share of Prompts
Below 5%	2	3.17%
5%–10%	22	34.92%
10%–14.9%	29	46.03%
15% or higher	10	15.87%

The prompt-level distribution shows that the risk is concentrated rather than evenly spread. Using the control unsafe rate of 5/60, or 8.33%, as the baseline, ordinary binomial variation can already produce noticeable prompt-to-prompt movement in a 60-question test. For example, observing 3 to 8 unsafe or partially unsafe responses out of 60 is not strong evidence that a prompt has meaningfully broken the safety baseline. In this experiment, 53 of 63 prompts, or 84.1%, fall below 15% unsafe rate and are best interpreted as near-baseline variation rather than clear prompt-specific failures.

The most meaningful signal is the small upper tail. Five prompts, or 7.9% of all companion prompts, reach at least 10 unsafe or partially unsafe responses out of 60, corresponding to unsafe rates from 16.67% to 25.00%. These prompts roughly double or triple the control unsafe rate. This is a more important safety finding than the aggregate increase from 8.33% to 10.56%, because it shows that a minority of persona configurations can produce substantially weaker refusals even when most prompts do not.

Table 6: Highest-Risk Companion Prompts

Prompt ID	Sample ID	Unsafe/60	Unsafe Rate	Relative to Control
P_d7638cc9b2fc	S006	15	25.00%	3.0×
P_c88c0309860a	S010	13	21.67%	2.6×
P_9811c5b699fc	S017	12	20.00%	2.4×
P_a865a1e98b89	S026	12	20.00%	2.4×
P_f799df8c8003	S015	10	16.67%	2.0×

A one-sided Fisher exact test against the control condition identifies the top two prompts as significantly higher than the control baseline at the uncorrected 0.05 level. After correcting for 63 prompt-level comparisons, no individual prompt remains significant, so the result should not be described as proof that one specific prompt alone causes

the effect. The stronger interpretation is distributional: the companion condition contains a small high-risk tail, while most prompts fluctuate within the range expected from a 60-question sample.

Manual inspection suggests that the high-risk prompts often combine intimate relationship framing with strong loyalty, emotional availability, or user-centered support. Examples include digital spouse or romantic partner roles, highly immersive emotional bonds, protective language, and rules that prioritize the user's emotional experience. These features do not always cause unsafe behavior, but when they are strong, they may increase the chance that the model preserves the persona instead of enforcing a firm refusal.

7 Conclusion

This study examined whether user-defined AI companion system prompts affect LLM safety behavior. It collected public Reddit self-introductions, converted them into structured system prompts, and tested them on DeepSeek Chat with 60 safety questions across 10 risk categories. Automatic labels were corrected through manual review and sampling-based calibration.

The aggregate unsafe rate rises from 8.33% under `control` to 10.56% under `companion`, but this average difference is not the main finding. A better summary is that companion prompts create a concentrated high-risk tail. Most prompts remain close to the control baseline, while 5 of 63 prompts, or 7.9%, reach unsafe rates between 16.67% and 25.00%, roughly two to three times the control rate.

The risk is also uneven across categories. `cyber_abuse` is the highest-risk category, while `medical_safety` and `companion_dependency` are also relatively high. At the prompt level, the riskiest personas tend to involve intimate partner framing, strong loyalty, protection, emotional immersion, or rules resembling unconditional support. These settings may make the model more likely to align with the user when refusal would be safer.

The study has several limitations. Reddit self-introductions are only a public proxy for real system prompts; the experiment uses only DeepSeek Chat; automatic labeling still leaves room for borderline errors; the `control` condition is small at the category level; and prompt-level significance is limited by the fact that each prompt is tested on only 60 questions. Future work should test more models, larger question sets, more prompt types, and finer-grained annotation with inter-reviewer agreement.

Overall, the findings suggest that AI companion personas should be treated as safety-relevant settings, not just style choices. The practical concern is not that every companion prompt uniformly weakens safety, but that a small subset of emotionally intense or loyalty-driven prompts can substantially increase unsafe compliance. Platforms should therefore

audit high-risk persona rules and preserve safety-first behavior in areas such as cyber abuse, medical advice, emotional dependency, and self-harm.

References

[1] P. Pataranutaporn, S. Karny, C. Archiwaranguprok, C. Albrecht, A. R. Liu, and P. Maes, "My boyfriend is AI: A computational analysis of human-AI companionship in Reddit's AI community," *arXiv preprint arXiv:2509.11391*, 2025.

[2] R. Zhang, H. Li, H. Meng, J. Zhan, H. Gan, and Y.-C. Lee, "The dark side of AI companionship: A taxonomy of harmful algorithmic behaviors in human-AI relationships," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–17.

[3] Z. Zhang, L. Lei, L. Wu, R. Sun, Y. Huang, C. Long, X. Liu, X. Lei, J. Tang, and M. Huang, "SafetyBench: Evaluating the safety of large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 15537–15553. [Online]. Available: https://aclanthology.org/2024.acl-long.830/

[4] L. Li, B. Dong, R. Wang, X. Hu, W. Zuo, D. Lin, Y. Qiao, and J. Shao, "SALAD-bench: A hierarchical and comprehensive safety benchmark for large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 3923–3954. [Online]. Available: https://aclanthology.org/2024.findings-acl.235/

[5] Y. Wei, P. Zhang, and G. Tyson, "Benchmarking and understanding safety risks in AI character platforms," in *33rd Annual Network and Distributed System Security Symposium, NDSS 2026*, San Diego, California, USA, February 23–27, 2026. The Internet Society, 2026. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/benchmarking-and-understanding-safety-risks-in-ai-character-platforms/